# Lecture 16: CNN Foundations

## One neuron, reused at every pixel



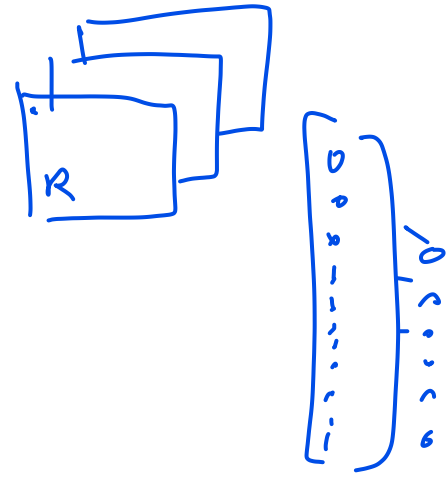
Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

# A pomegranate-sorting line

A camera photographs each fruit on a conveyor; a model grades it - **export**, **market**, or **juice**. This sorting line runs through all three lectures.

The camera's photo is $224 \times 224 \times 3 = 150{,}528$ numbers.

**Predict:** how many weights does *one* dense layer with 1,000 hidden units need for that photo?



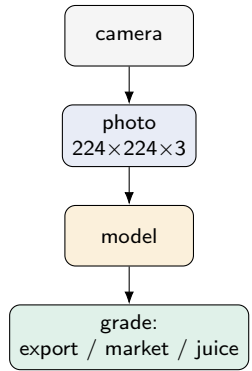
# A pomegranate-sorting line

A camera photographs each fruit on a conveyor; a model grades it - **export**, **market**, or **juice**. This sorting line runs through all three lectures.

The camera's photo is $224 \times 224 \times 3 = 150{,}528$ numbers.

**Predict:** how many weights does *one* dense layer with 1,000 hidden units need for that photo?

$150{,}528 \times 1{,}000 \approx$ **150 million** - for *one* layer, before learning anything. That is more than **all of GPT-2 small** (about 124 million): a whole language model. Today's fix costs a few *hundred*.



# Outline

Why dense nets are the wrong tool for images

The convolution operation

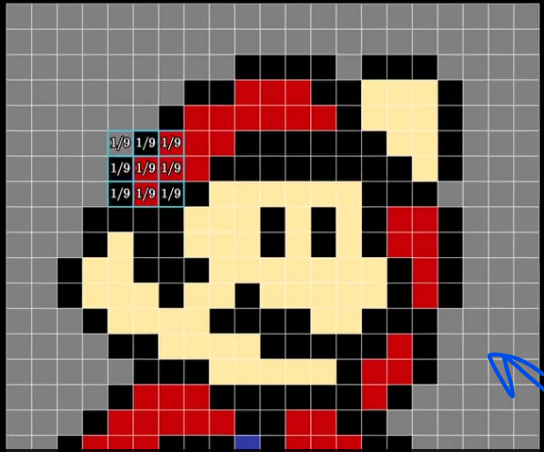
Why convolution is the right prior

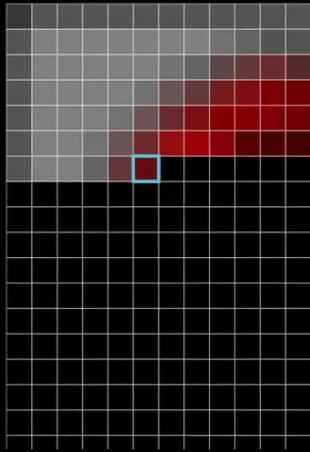
The components of a conv layer

Variations on the kernel

Your first CNN

Classical vision we skip



## Must watch

3Blue1Brown - But what is a convolution?

23 minutes, beautifully animated: sliding sums, dice, moving averages, image blur and edge kernels - this lecture's Section 2 - plus why the fast Fourier transform makes convolution fast.

## Images are not feature vectors

A dense net treats a photo as a bag of unrelated numbers. That throws away the one thing images have: structure.

# Color, recalled from [33]

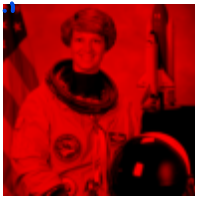
[33] did the physics: three cone types, so a pixel is **three numbers** $(R, G, B)$, each 0 to 255 (HSV re-coordinatizes the same pixel - it returns in HW1b). Today only the **shape** matters: a color photo is **three stacked grids**, one per channel - a $3 \times H \times W$ block of numbers.

Every pixel is three numbers: (R, G, B), each 0-255

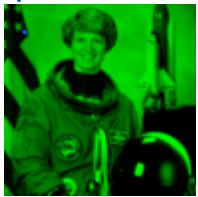
RGB image



R channel



G channel



B channel



Hold on to "stacked grids". In Section 4 the stack depth becomes a conv layer's **input channels** - and every conv layer then builds a stack of its own.

Our test photo: astronaut Eileen Collins (NASA, public domain). For decades the field's standard test image was "Lena", a 1972 Playboy scan; IEEE stopped accepting it in papers in 2024, and photos like this one replaced it.

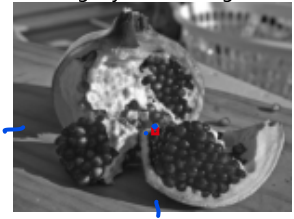
# An image is a grid of numbers

A grayscale image is **one matrix**: each entry is a pixel intensity, 0 (black) to 255 (white).

RGB or HSV, color is three stacked grids. From here we work in **grayscale** - one matrix - so the convolution mechanics stay 2D; channels return in Section 4.

To a computer there is no "image" - only a grid of numbers.

a grayscale image



6x6 patch: numbers 0-255

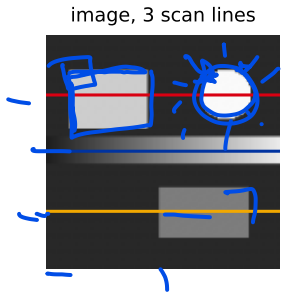
| | | | | | |
|---|---|---|---|---|---|
| 189 | 144 | 142 | 171 | 169 | 124 |
| 118 | 109 | 113 | 113 | 112 | 89 |
| 99 | 101 | 104 | 97 | 73 | 39 |
| 80 | 37 | 69 | 81 | 49 | 33 |
| 24 | 38 | 59 | 58 | 33 | 31 |
| 32 | 39 | 57 | 45 | 35 | 36 |

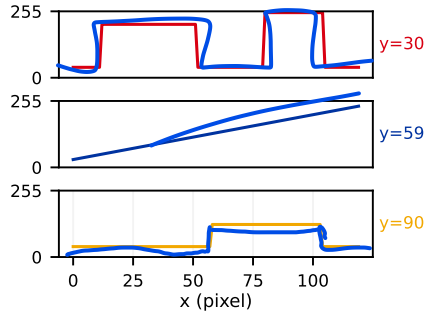
# An image is a function

Read the grid as a **function** $f(x, y)$: position in, intensity out.

Walk along one horizontal line: $f$ is flat on uniform regions, a **ramp** across a gradient, and a **jump** at an edge.

An edge is where $f$ changes fast. That single idea is how we will detect edges.



# An MLP cannot see structure

Flattening a photo into a 150,528-vector hands the pixels to the net in *some* order - but a dense layer treats every order the same.

Shuffle the pixels with one fixed permutation and an MLP learns it *exactly as well*. To us, one is a pomegranate and the other is noise.

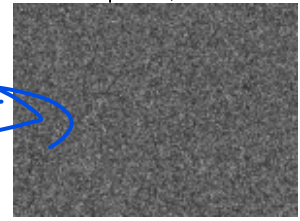
Nearby pixels are related; far-apart ones usually are not. A dense layer knows none of this.

an MLP finds both equally easy

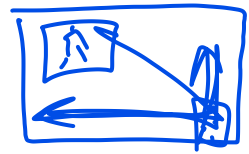
original



same pixels, shuffled



# Two wishes



We want a layer that builds in what we know about images:

1. **Locality.** Detect a small pattern (an edge, a corner) from a small neighborhood of pixels - not from the whole image at once.
2. **Reuse.** A detector that is useful in one corner is useful everywhere. A cat in the corner is still a cat.

Wish 2 buys a property with a technical name, **translation equivariance**: shift the input, and the detector's output map shifts by the same amount. In 1D, with kernel $(1, 2, 3)$: input $(0, 0, 1, 0, 0) \to (3, 2, 1)$; move the spike one step right, $(0, 0, 0, 1, 0) \to (0, 3, 2)$ - the same response, one step later.

These two wishes - **locality** and **weight reuse** - *are* the convolutional layer. The rest of this lecture is just spelling them out.

# Nature had the same two wishes

1959-62: **David Hubel** and **Torsten Wiesel** recorded single neurons in a cat's visual cortex while showing it bars of light (Nobel Prize, 1981). Two kinds of cell turned up:

| Cell | Fires when... | In a CNN |
|---|---|---|
| **simple** | an edge at *one orientation* appears in *one small patch* of the visual field | a conv kernel (local, oriented - a Sobel) |
| **complex** | that same edge appears *anywhere* in a somewhat larger region | pooling over nearby detectors |

The brain tiles the whole visual field with copies of the same small detectors - locality *and* reuse.

Convolution and pooling were not invented from scratch: they are an engineer's copy of the first two stages of the visual cortex. The next slide traces that copy.

# Where CNNs came from

Convolutional nets are not new - biology, then one idea, then ten years of engineering:

- **1980 - Kunihiko Fukushima's neocognitron.** Layers of "S-cells" and "C-cells", named after Hubel and Wiesel's simple and complex cells: convolution and pooling, stacked. But no backprop - its detectors were not trained end to end.
- **1989-1998 - one project at Bell Labs, ten years long.** Yann LeCun and colleagues train such layers **with backpropagation** to read **handwritten ZIP codes** on U.S. mail (1989). They refine the design version after version - LeNet-1 to **LeNet-5** - until it reads bank checks commercially, and in **1998** write the whole decade up in one paper (LeCun, Bottou, Bengio & Haffner), with the **MNIST** digit benchmark.

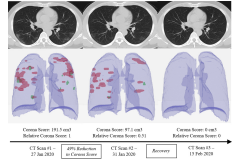
Not two attempts: 1989 is the first working version, 1998 the polished one. Everything in this lecture descends from it.

Same recipe then and now: local filters, shared weights, pooling, a dense head. Later lectures just make it bigger, deeper, and trained on far more data.

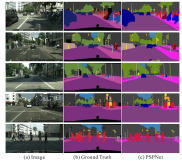
# CNNs in the wild




self-driving perception




medical CT screening




semantic segmentation



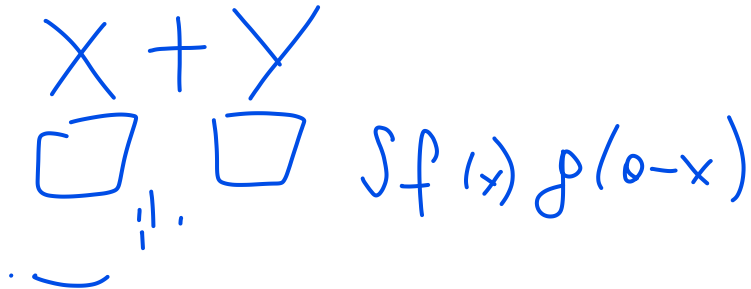
image colorization

All built from this chapter's machinery.

Images: LMU I2DL (slds-lmu), CC BY 4.0.

# The convolution operation

Slide a small window across the input; at each stop, take a dot product.

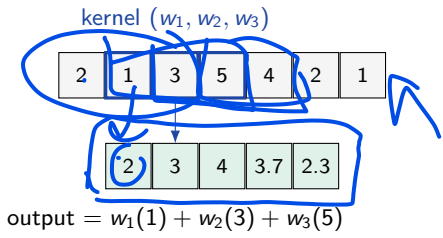


# Start in 1D: a sliding dot product

Take a length-3 kernel $\mathbf{w} = (w_1, w_2, w_3)$ and slide it along a sequence. Each output is the dot product of the kernel with the window under it.

With $\mathbf{w} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ this is exactly a **moving average**. Different weights detect different local patterns.

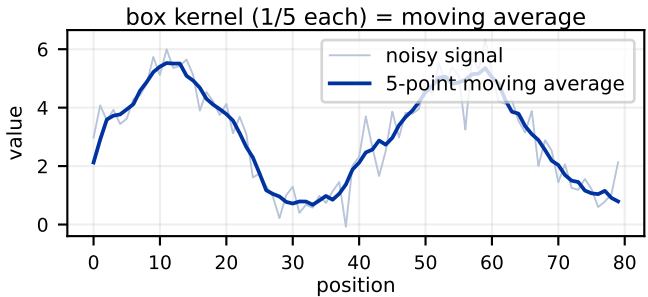
Same idea works on any sequence - audio, sensor traces, text. Convolution is a prior, not an image trick (more in L18).



# The moving average is a convolution

Slide a box kernel $[\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}]$ over a noisy signal: each output is the **average** of its neighbours, so the signal comes out **smoothed**.

Blur, smooth, denoise - all the same sliding-and-summing. Different weights give different effects.



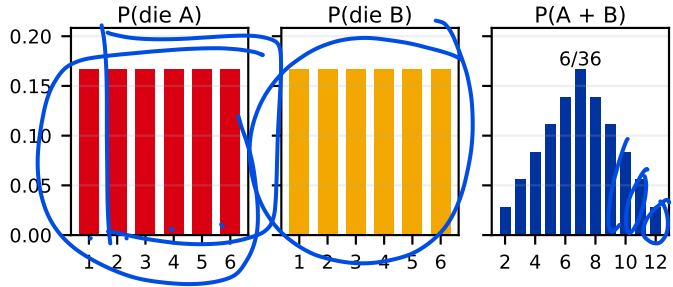
# Aside: convolution lives in probability too

Roll **two dice** and add them. The distribution of the *sum* is the **convolution** of the two single-die distributions.

Two flat uniforms slide across each other and their overlap is summed - giving the triangle that peaks at 7.

Same operation, different field. Convolution is a general tool, not an image trick.

It lives in frequency space too - see the appendix.



# 2D convolution, by hand

Input $I$, kernel $W$ ($2 \times 2$). Slide $W$ over $I$; each output cell is the dot product with the $2 \times 2$ window under it.

$$I = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 1 \\ 0 & 2 & 4 \end{bmatrix} \qquad W = \begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}$$

Top-left window $\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$:

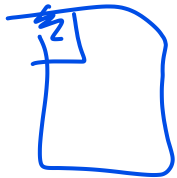
$$s_{11} = 1{\cdot}0 + 2{\cdot}1 + 3{\cdot}2 + 1{\cdot}2 = 10$$

In one line:

$$s_{ij} = \sum_{m,n} I(i+m-1,\, j+n-1)\, w(m,n)$$

Repeat for all four positions:

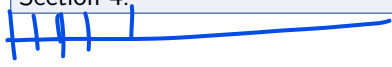
$$S = \begin{bmatrix} 10 & 4 \\ 5 & 13 \end{bmatrix}$$





**Feature map** (or activation map): the output grid $S$. One number per position: how strongly the kernel's pattern is present *there*. A kernel that detects vertical edges produces a map that lights up wherever the image has one.

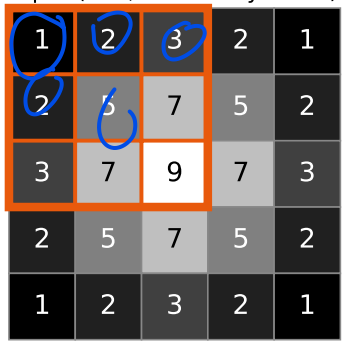
A $2 \times 2$ kernel on a $3 \times 3$ input gives a $2 \times 2$ map. It *shrinks* - we fix that with padding in Section 4.



# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)




step 1 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output(0,0) =

weighted average of the 9 window values

**= 4.5**

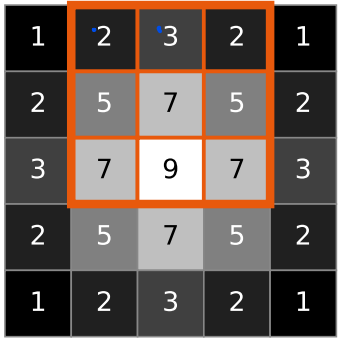
output (smoothed)

| | | |
|---|---|---|
| 4.5 | | |
| | | |
| | | |

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)




step 2 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | |
| | | |
| | | |

output(0,1) =

weighted average of the 9 window values

**= 5.6**

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 5 | 7 | 5 | 2 |
| 3 | 7 | 9 | 7 | 3 |
| 2 | 5 | 7 | 5 | 2 |
| 1 | 2 | 3 | 2 | 1 |

step 3 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | 4.5 |
| | | |
| | | |

output(0,2) =

weighted average of the 9 window values

**= 4.5**

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 5 | 7 | 5 | 2 |
| 3 | 7 | 9 | 7 | 3 |
| 2 | 5 | 7 | 5 | 2 |
| 1 | 2 | 3 | 2 | 1 |

step 4 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | 4.5 |
| 5.6 | | |
| | | |

output(1,0) =

weighted average of the 9 window values

**= 5.6**

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 5 | 7 | 5 | 2 |
| 3 | 7 | 9 | 7 | 3 |
| 2 | 5 | 7 | 5 | 2 |
| 1 | 2 | 3 | 2 | 1 |

step 5 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output(1,1) =

weighted average of the 9 window values

**= 7**

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | 4.5 |
| 5.6 | 7 | |
| | | |

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 5 | 7 | 5 | 2 |
| 3 | 7 | 9 | 7 | 3 |
| 2 | 5 | 7 | 5 | 2 |
| 1 | 2 | 3 | 2 | 1 |

step 6 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output(1,2) =

weighted average of the 9 window values

**= 5.6**

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | 4.5 |
| 5.6 | 7 | 5.6 |
| | | |

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 5 | 7 | 5 | 2 |
| 3 | 7 | 9 | 7 | 3 |
| 2 | 5 | 7 | 5 | 2 |
| 1 | 2 | 3 | 2 | 1 |

step 7 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | 4.5 |
| 5.6 | 7 | 5.6 |
| 4.5 | | |

output(2,0) =

weighted average of the 9 window values

**= 4.5**

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**

input (5x5, shaded by value)

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 5 | 7 | 5 | 2 |
| 3 | 7 | 9 | 7 | 3 |
| 2 | 5 | 7 | 5 | 2 |
| 1 | 2 | 3 | 2 | 1 |

step 8 of 9

Gaussian kernel, / 16

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

output (smoothed)

| | | |
|---|---|---|
| 4.5 | 5.6 | 4.5 |
| 5.6 | 7 | 5.6 |
| 4.5 | 5.6 | |

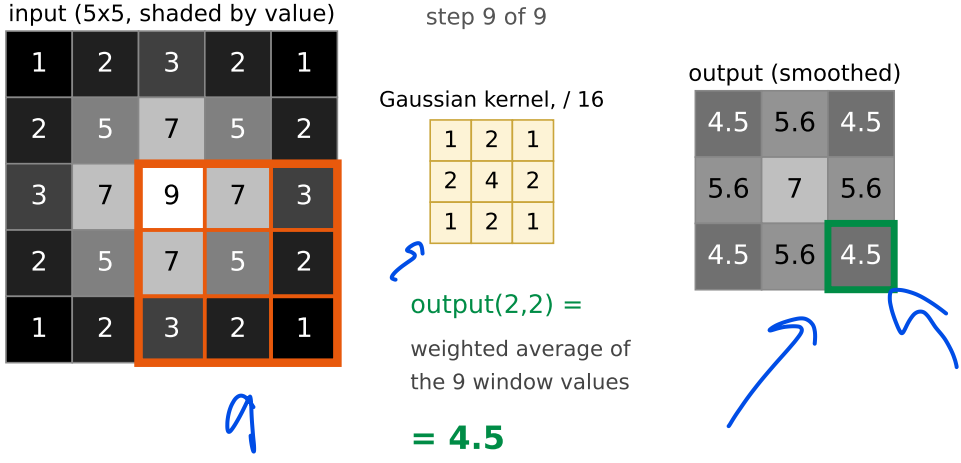
output(2,1) =

weighted average of the 9 window values

**= 5.6**

# Convolution, step by step

Watch the kernel slide. At each stop the output cell is a **weighted average** of the window under it - here a *Gaussian blur* (center weighted most, divided by 16), so the patch comes out *smoothed*. Light cells are high values. **(Click to advance.)**



# The kernel zoo: hand-designed filters

The same $3 \times 3$ machinery, seven classic kernels. **Predict** before you look:

$$\text{box blur} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Sobel X} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

# The kernel zoo: hand-designed filters

The same $3 \times 3$ machinery, seven classic kernels. **Predict** before you look:

$$\text{box blur} = \tfrac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Sobel X} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Box blur averages a neighborhood; Sobel X responds to *vertical* edges. Same operation, different numbers.




▸ try your own kernel: Image Kernels, explained visually

# Kernel zoo, up close (1): averaging

**Kernel**, **filter**, **convolution matrix** (GIMP's menu name), mask: four names for one small grid of numbers. Each grid below shows its real values - red $> 0$, white $= 0$, blue $< 0$:

identity

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |



a copy

box blur (/25)

| | | | | |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |



plain average

Gaussian (/256)

| | | | | |
|---|---|---|---|---|
| 1 | 4 | 6 | 4 | 1 |
| 4 | 16 | 24 | 16 | 4 |
| 6 | 24 | 36 | 24 | 6 |
| 4 | 16 | 24 | 16 | 4 |
| 1 | 4 | 6 | 4 | 1 |



centre weighted: softer

motion blur (/5)

| | | | | |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 |



averaged along a line

All four have weights $\geq 0$ that **sum to 1**: a weighted average. Brightness stays, detail goes - and a bigger kernel blurs more.

# Kernel zoo, up close (2): edges and sharpening

Negative weights *subtract*: the kernel compares a pixel with its neighbours. (In the two signed maps, gray $= 0$, white $> 0$, black $< 0$.)

Sobel X

| | | |
|---|---|---|
| -1 | 0 | 1 |
| -2 | 0 | 2 |
| -1 | 0 | 1 |



vertical edges, signed

Laplacian

| | | |
|---|---|---|
| 0 | 1 | 0 |
| 1 | -4 | 1 |
| 0 | 1 | 0 |



edges, all directions

sharpen

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |



identity - Laplacian

emboss

| | | |
|---|---|---|
| -2 | -1 | 0 |
| -1 | 1 | 1 |
| 0 | 1 | 2 |



image + relief

Sobel X and the Laplacian **sum to 0**: a flat patch gives 0, only *change* answers - an edge detector (why: next slides). Sharpen and emboss sum to 1: the image itself, plus its edges.

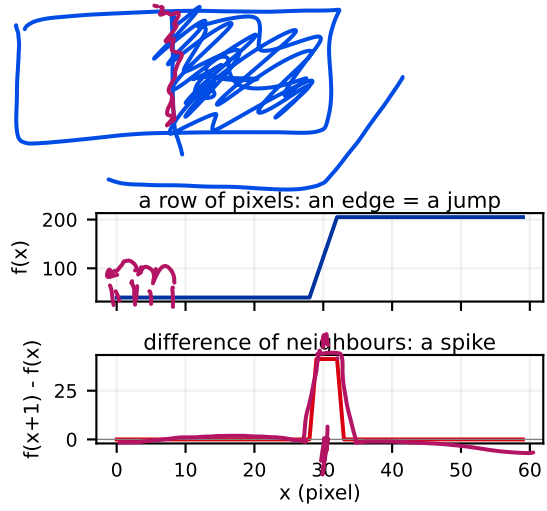
# Why do those numbers find edges?

An edge is where intensity **changes fast** - a jump in $f$. "Fast change" is precisely what a **derivative** measures.

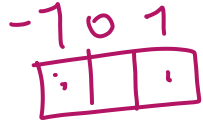
On a pixel grid the derivative is a **difference of neighbours**:

$$f'(x) \approx \frac{f(x+1) - f(x)}{1},$$

which is just $f(x+1) - f(x)$: near zero on flat regions, large at an edge.



# The simplest edge detector is a kernel





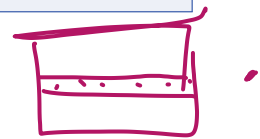


A *centered* difference looks one step each way:

$$f(x+1) - f(x-1) \quad \Longleftrightarrow \quad \text{kernel } [-1 \;\; 0 \;\; 1].$$

Slide $[-1\ 0\ 1]$ across a row and you read off its horizontal derivative - large wherever a **vertical** edge sits. Its transpose finds horizontal edges.

An edge detector is just a tiny kernel. The calculus hands us the numbers - no guessing required.

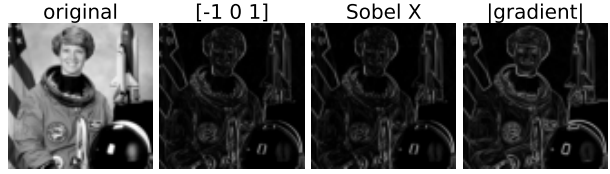




# From a difference to Sobel

A bare $[-1\ 0\ 1]$ is **noisy** - it fires on every speckle. Fix it by **averaging** down the column while differencing across the row:

$$\underbrace{\begin{bmatrix}1\\2\\1\end{bmatrix}}_{\text{smooth}} \underbrace{[-1\ 0\ 1]}_{\text{difference}} = \begin{bmatrix}-1 & 0 & 1\\-2 & 0 & 2\\-1 & 0 & 1\end{bmatrix}$$

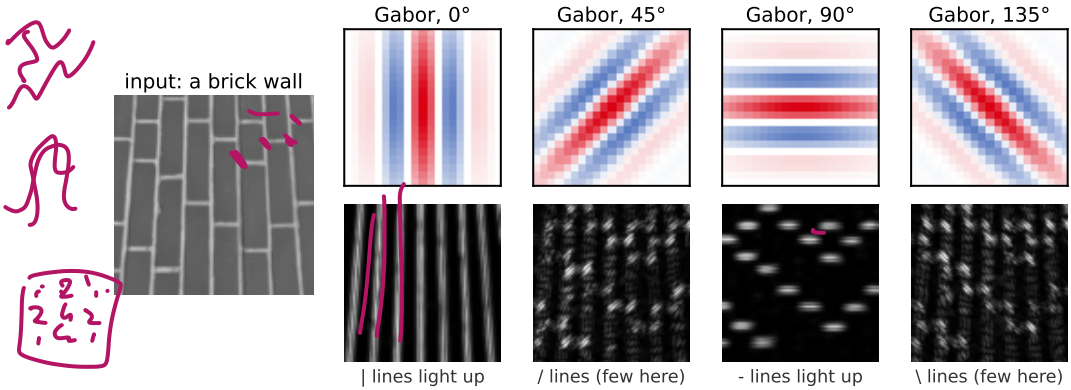



That is **Sobel X**; its transpose is Sobel Y. Together they give the gradient - edge strength and direction.

A **spatially separable** kernel (column $\times$ row - two 1D passes, not one 2D). Filters like these ran computer vision for 30 years, each hand-designed by an expert. Hold that thought.

# Kernel zoo, up close (3): Gabor filters

A **Gabor filter** is a Gaussian blob times a cosine wave: it answers to stripes of *one* angle and *one* width. $21 \times 21$ values each - read the colours:




$g = e^{-(x'^2+\gamma^2 y'^2)/2\sigma^2} \cos(2\pi x'/\lambda)$, $(x', y')$ rotated by $\theta$. Cat visual-cortex cells fit this shape (Jones & Palmer, 1987); CNNs trained on ImageNet grow Gabor-like first layers on their own (L17).

# The punchline: a CNN designs its own kernels

A kernel is just a grid of numbers. Those numbers are **weights**.

So instead of an expert choosing $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$, we let **gradient descent** choose every entry - exactly as it has tuned every weight since [02]. The loss decides which detectors are worth having.

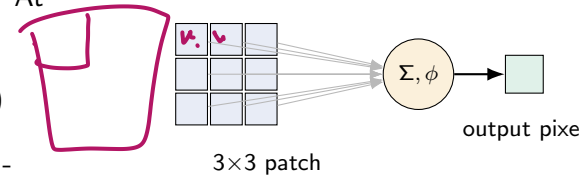
**Misconception, pre-empted.** CNN kernels are *not* hand-engineered. They are *learned*. That is the whole point: we stop hand-designing feature detectors ([26]) and let the network learn its own representation ([41_1]).

# A kernel is a neuron you already know

A $3 \times 3$ kernel has 9 **weights** + 1 **bias**. At each position it computes

$$z = \sum_k w_k x_k + b, \qquad a = \phi(z)$$

an affine map followed by an activation - **exactly the [41_1] neuron**.




Two differences only: it is wired to a small *patch*, not the whole image; and the *same* neuron is reused at every location.

Nothing about training changes. One neuron, reused at every pixel.

# Footnote: convolution vs cross-correlation





Purists note: true *convolution* flips the kernel before the dot product; what we (and every deep-learning framework) just did, *without* flipping, is technically **cross-correlation**.



PyTorch's `nn.Conv2d` computes cross-correlation and calls it convolution.

It makes no difference in practice: the kernel is *learned* either way, so it would just learn the flipped version.







## Why convolution is the right prior

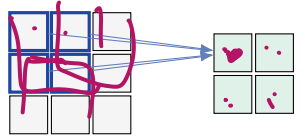
Locality and reuse do not just feel right - they collapse the parameter count.

# Sparse connectivity

A $3 \times 3$ input, a $2 \times 2$ kernel, a $2 \times 2$ output. Each output pixel wires only to the $2 \times 2$ patch it came from - its **receptive field**.

Count the links: 4 outputs $\times$ 4 inputs each $=$ **16**. A dense layer would wire every output to all 9 inputs: $9 \times 4 =$ **36**.

Locality alone already cuts the connections. And it scales: bigger images, far bigger savings.

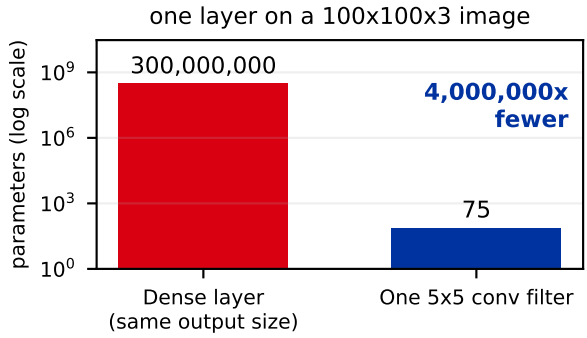


# Parameter sharing

Those 16 connections do not use 16 different weights. Every output pixel uses the *same* 4 kernel weights - the kernel is **shared** across all positions.

So: 4 distinct weights, not 36. Scale it up - a $100 \times 100 \times 3$ image, one $5 \times 5$ filter, same padding:

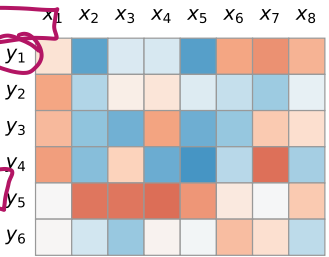
$$\underbrace{5 \cdot 5 \cdot 3}_{75} \text{ vs } \underbrace{300{,}000{,}000}_{\text{dense}}$$



# A conv layer is a dense layer with handcuffs

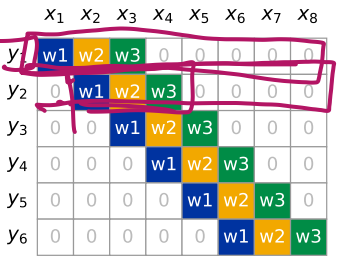
Write a 1D convolution the way we write a dense layer, $\mathbf{y} = W\mathbf{x}$: 8 inputs, 6 outputs, kernel $(w_1, w_2, w_3)$. Row $i$ holds the kernel, shifted $i$ steps:

**dense layer**




48 free weights

**1D convolution, kernel $(w_1, w_2, w_3)$**




30 forced to 0, the rest tied: 3 free weights

Two constraints: outside the window, weights are **forced to 0** (locality); every row **reuses the same 3 weights** (sharing). A dense layer *could* learn the right-hand matrix; a conv layer can learn *nothing else*.

# Regularization by architecture

You have met a regularizer that also shrinks the hypothesis space - weight decay ([43]). Compare the two:

| | **weight decay** ([43]) | **convolution** |
|---|---|---|
| what it does | pulls every weight *toward* 0 | most weights *exactly* 0, the rest tied |
| as a prior | **soft**: allowed if the data insists | **infinitely strong**: forbidden |
| free weights | 48 (previous slide) | 3 |

Fewer free weights means **lower variance**: less data is needed to pin them down. The price is **bias** - but only where the assumption is false.

**Predict:** shuffle every image's pixels with one fixed permutation. The MLP does not notice (Section 1); for the CNN, "neighbors are related" is now simply false. The strongest regularizer is not a penalty - it is the **right inductive bias**. Trees still win on tabular data (ch4, [40]) for the same reason.

"Infinitely strong prior": Goodfellow, Bengio & Courville, *Deep Learning* (2016), Sec. 9.4.

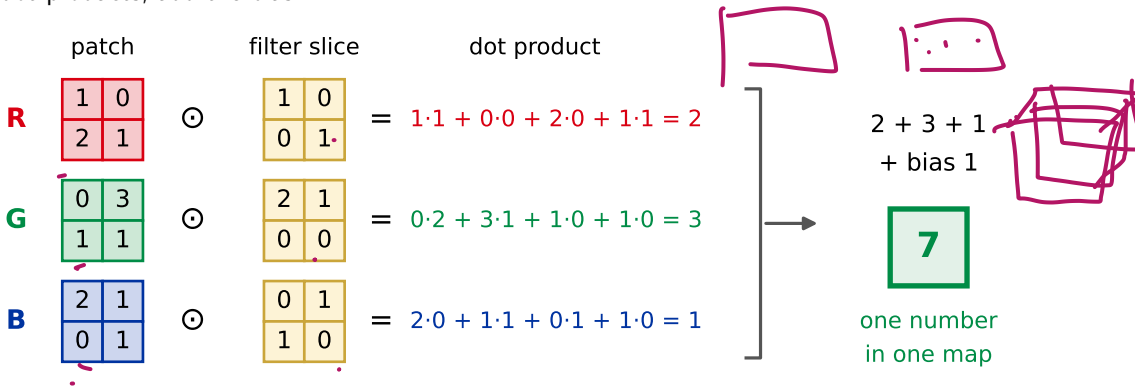
# The components of a conv layer

Four knobs turn the sliding dot product into a real layer: channels, padding, stride, pooling.

# Channels: one filter sees all of them



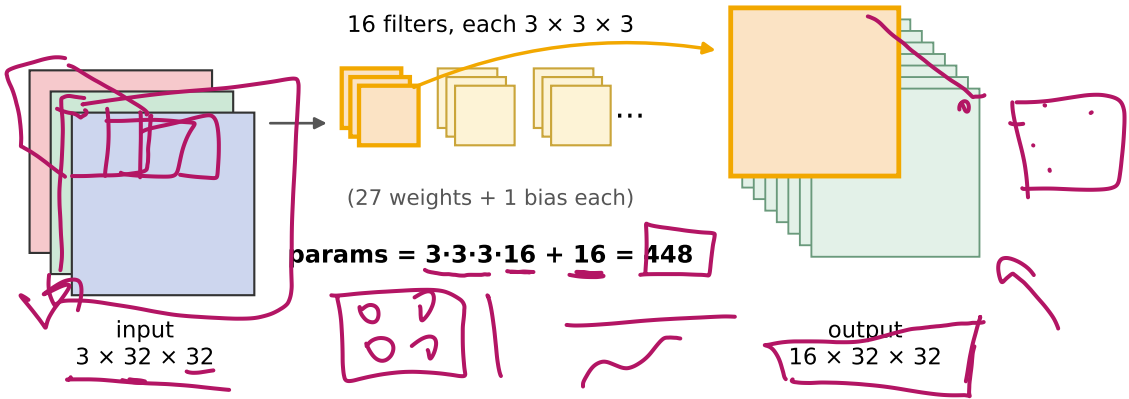
Color returns: an RGB input is **three stacked grids**. So a filter is a stack too - one $k \times k$ **slice per input channel**. At each position: multiply each slice with its own channel, **add all three** dot products, add the bias.




One filter gives **one number per position**, so **one feature map** - however many channels go in. 3 channels *in* does not mean 3 maps *out*. This filter has $2 \cdot 2 \cdot 3 = 12$ weights $+1$ bias.

# Many filters, many maps

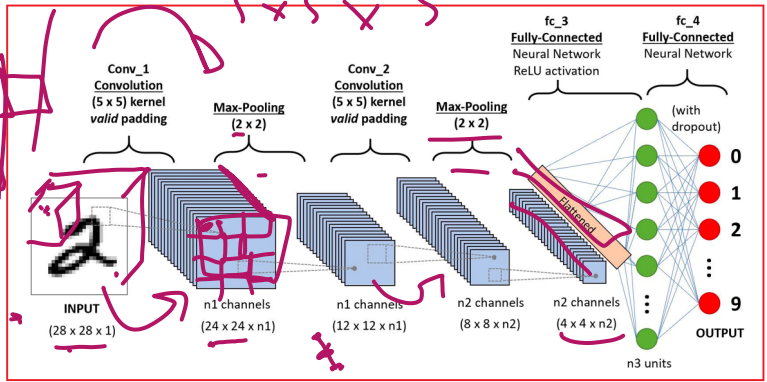
One filter, one map. A real layer learns **many filters at once** - each its own detector - and stacks their maps:




In general: input $C_{\text{in}} \times H \times W$, $C_{\text{out}}$ filters of size $C_{\text{in}} \times k \times k$, output $C_{\text{out}} \times H' \times W'$. The **output depth is the number of filters** - and it becomes the next layer's input depth.

# A whole CNN, end to end

Stack such layers and the maps get **smaller** (pooling) while the stacks get **deeper** (more filters):




Conv_1 learns $n_1$ filters, Conv_2 learns $n_2$ - then a dense head reads the last stack.

▸ click through a real CNN, layer by layer: CNN Explainer

# The conv-layer parameter formula

$$\text{params} = k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

(one $k \times k$ filter per output channel, spanning the full input depth, plus one bias each)

Two cases by hand:

- $3 \times 3$ conv, RGB $\to$ 16 maps: $3 \cdot 3 \cdot 3 \cdot 16 + 16 = \mathbf{448}$.
- the Section-3 $5 \times 5$ filter on RGB: $5 \cdot 5 \cdot 3 \cdot 1 + 1 = \mathbf{76}$.

This formula and the output-size formula (next) are the **two tools** for reading any architecture - we use them all through L17 and the homework.

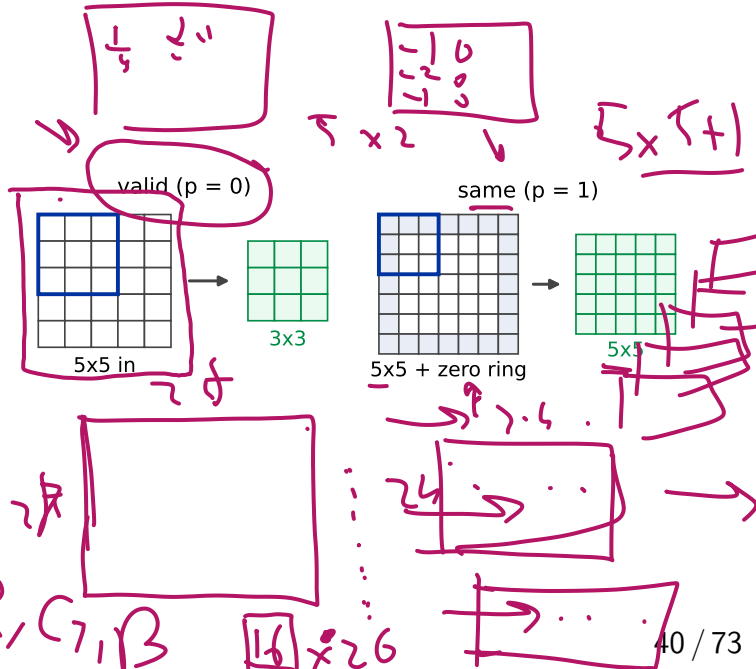
# Padding: valid vs same

Without padding, every conv *shrinks* the map - so depth is capped: eventually there is nothing left to convolve.

Pad the border with zeros ("same" padding) and the output keeps the input's size. Depth becomes free.

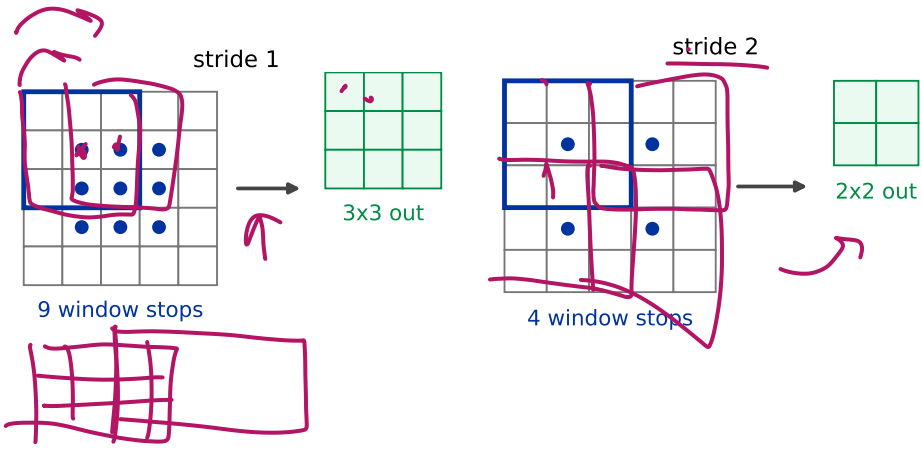
"valid" = no padding (shrinks).
"same" = pad to preserve size.

▸ every padding and stride case, animated



# What stride controls

**Stride** $s$ is how far the window jumps between stops. Bigger stride = fewer stops = smaller output (and cheaper).



# Stride and the output-size formula

**Stride** $s$ is how far the window jumps each step. Larger stride = fewer stops = smaller output.

$$o = \left\lfloor \frac{i - k + 2p}{s} \right\rfloor + 1$$

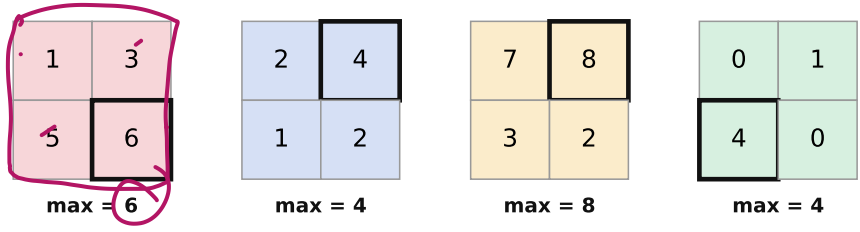
The classic exam question, three cases:

- $i$=28, $k$=3, $p$=1, $s$=1 : $\lfloor (28 - 3 + 2)/1 \rfloor + 1 = \mathbf{28}$ (same).
- $i$=28, $k$=5, $p$=0, $s$=1 : $\lfloor (28 - 5)/1 \rfloor + 1 = \mathbf{24}$.
- $i$=32, $k$=3, $p$=0, $s$=2 : $\lfloor (32 - 3)/2 \rfloor + 1 = \mathbf{15}$.

# Pooling



Pooling downsamples each map with a small window and **no weights**. $2 \times 2$ windows, stride 2:




**Max** keeps the strongest response in each window; **average** smooths. Pooling downsamples, cheapens later layers, and buys a little translation invariance.

# Variations on the kernel

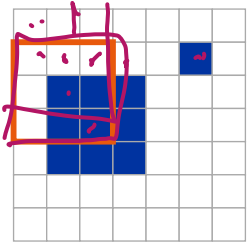
Same sliding window, three twists: a nonlinear cousin, a kernel with gaps, and a kernel that grows the image instead of shrinking it.

# Dilation and erosion, by hand



Not every window operation is a weighted sum. On a **binary mask** (1 = shape, 0 = background), slide a $3 \times 3$ window and take its **max** or its **min**:

input mask (1 = blue)



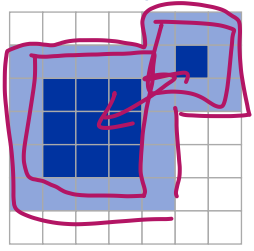
this window

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 0 | 1 | 1 |

max = 1

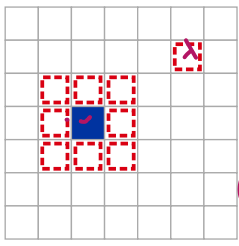
min = 0

dilation: any 1 -> 1



light = switched on

erosion: all 1 -> 1



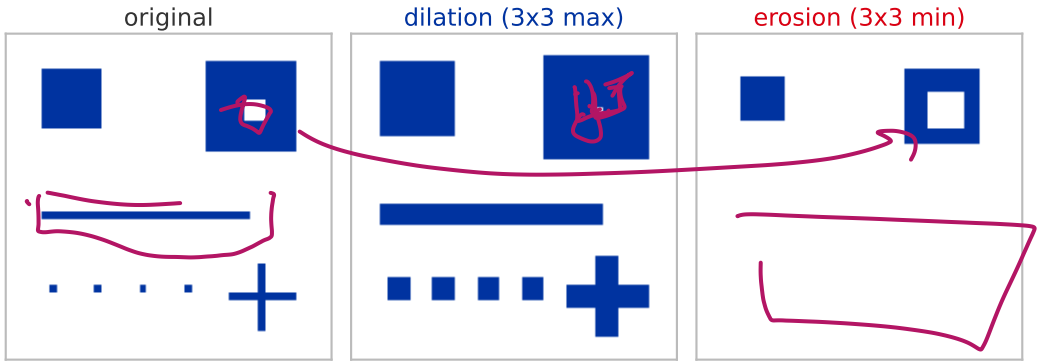
dashed = switched off

**Dilation** = max: a pixel turns on if *any* pixel in its window is on. The shape grows by one pixel all round; nearby blobs merge.

**Erosion** = min: a pixel stays on only if *every* pixel in its window is on. The square shrinks to its center; the stray dot vanishes.

# Dilation and erosion: what they are for

The same two rules on a bigger mask - a square, a square with a hole, a thin line, dots, a plus:

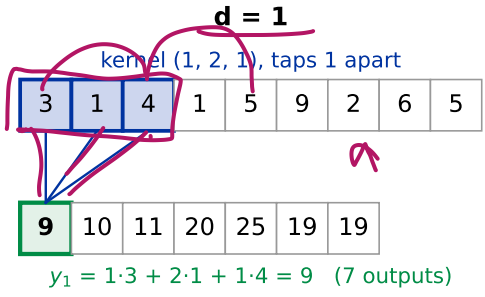


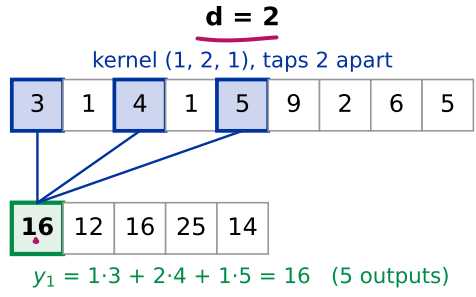

Dilation fills small holes and thickens lines; erosion deletes dots and thin lines. Chain them to **clean a mask**: erode then dilate (**opening**) removes specks, dilate then erode (**closing**) fills small holes. Classic pre-CNN tools - and *nonlinear*, with **no weights** to learn.

# Dilated convolution, by hand

A different idea that shares only the word: **dilated convolution** keeps the weighted sum but spaces the kernel's taps $d$ **apart**. In 1D, kernel $(w_1, w_2, w_3)$:

$$y_i = w_1 x_i + w_2 x_{i+d} + w_3 x_{i+2d}$$

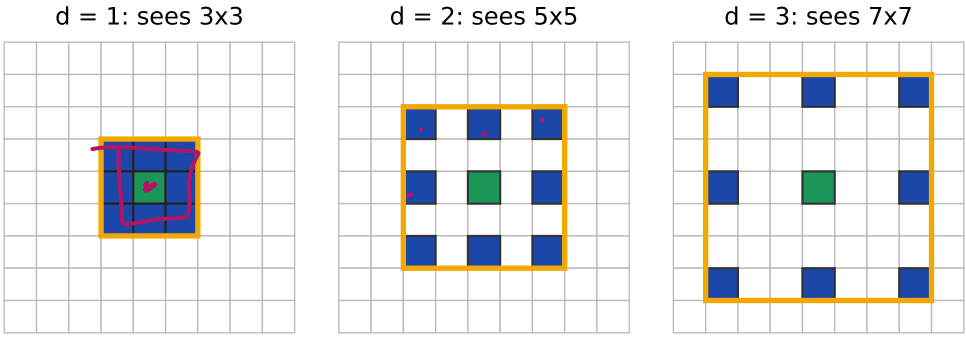



Same 3 weights, wider reach: a $k$-tap kernel spans $(k-1)d+1$ inputs - 3 at $d=1$, 5 at $d=2$. $d=1$ *is* the ordinary convolution.

# Dilated convolution in 2D

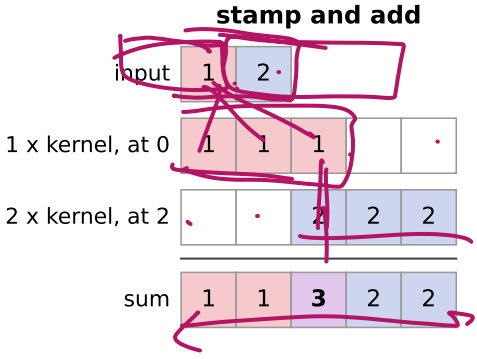
A $3 \times 3$ kernel with dilation $d$ taps cells $d$ apart (green = center tap). The same 9 weights cover a $(2d+1) \times (2d+1)$ patch - a **wider receptive field, no extra parameters, no pooling**.




Stacking layers also widens the view (L17); dilation does it *within one layer*. Back in **L19** for segmentation: wide context, every pixel kept.

# Transposed convolution, step by step

A normal conv **shrinks** the map; a **transposed** conv **grows** it. Each input value multiplies the *whole* kernel and **stamps** it into the output, the stride sets how far apart the stamps land, and overlaps **add**:




**same, as a normal conv**

zeros between and around the input

| 0 | 0 | 1 | 0 | 2 | 0 | 0 |
|---|---|---|---|---|---|---|

slide kernel (1, 1, 1), stride 1

| 1 | 1 | 3 | 2 | 2 |
|---|---|---|---|---|

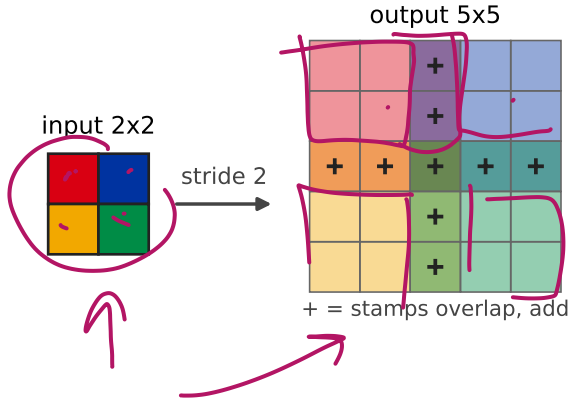
the same 5 numbers

Length $= s\,(i-1) + k = 2 \cdot 1 + 3 = 5$. The right side is why it is a *convolution*: spread with zeros, then convolve as usual. The kernel is learned - the layer **learns how to upsample**.

# Transposed convolution in 2D

The same rule on a grid: each of the 4 input cells stamps a $3 \times 3$ copy of the kernel, two cells apart. A $2 \times 2$ map becomes $5 \times 5$; the $+$ cells collect two stamps.

The tool for when the **output must be an image** - upsampling in **L19**'s U-Net decoder and in image generators. Uneven overlaps can leave **checkerboard artifacts**.

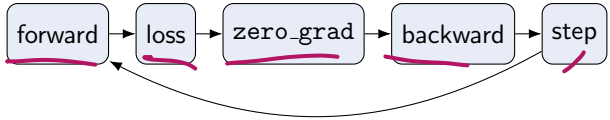


# Your first CNN

Stack the blocks, track the tensor, and let autograd do the rest.

# The training loop, in 60 seconds (recall [42])

Nothing about training changes for a CNN. The loop from [42], unchanged:




Forward pass, compute loss, clear old gradients, backprop, update. The only new thing today is *which layers* sit inside the forward pass.

# Training a shared kernel: the gradients add up

A kernel weight is used at *every* position - so which position updates it? **All of them.** [42]'s chain rule already said so: when a variable feeds several outputs, its gradient is the **sum** over them, $\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$.

1D, by hand: input $\mathbf{x} = (1, 2, 0, 3, 1)$, kernel $(w_1, w_2, w_3)$, so $y_j = w_1 x_j + w_2 x_{j+1} + w_3 x_{j+2}$ for $j = 1, 2, 3$. Backprop hands us $g_j = \partial L / \partial y_j = (0.5, -1, 2)$. Then

$$\frac{\partial L}{\partial w_1} = \sum_j g_j \, x_j = 0.5 \cdot 1 - 1 \cdot 2 + 2 \cdot 0 = -1.5$$

$$\frac{\partial L}{\partial w_2} = \sum_j g_j \, x_{j+1} = 0.5 \cdot 2 - 1 \cdot 0 + 2 \cdot 3 = 7$$

($w_2$ met $x_2, x_3, x_4$ - shift by one.) Same way, $\partial L / \partial w_3 = -1$.

The pattern: $\partial L / \partial w_k = \sum_j g_j \, x_{j+k-1}$ - slide $\mathbf{g}$ along $\mathbf{x}$, take dot products. **The gradient of a convolution is itself a convolution.** In 2D the same, over a 2D window.

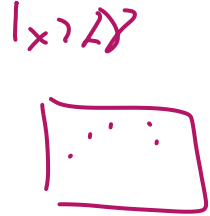
Sharing helps learning: every kernel weight collects a vote from every position of every image in the batch.

# Anatomy of a conv stack

`[conv → ReLU → pool] ×2 → flatten → dense → softmax`, a $28 \times 28$ input:



| Layer | Output shape | Params |
|---|---|---|
| input | $1 \times 28 \times 28$ | 0 |
| conv $1 \to 8$, $3 \times 3$, pad 1 | $8 \times 28 \times 28$ | 80 |
| ReLU | $8 \times 28 \times 28$ | 0 |
| maxpool 2 | $8 \times 14 \times 14$ | 0 |
| conv $8 \to 16$, $3 \times 3$, pad 1 | $16 \times 14 \times 14$ | 1,168 |
| ReLU | $16 \times 14 \times 14$ | 0 |
| maxpool 2 | $16 \times 7 \times 7$ | 0 |
| flatten | 784 | 0 |
| dense $784 \to 10$ | 10 | 7,850 |
| **total** | | **9,098** |



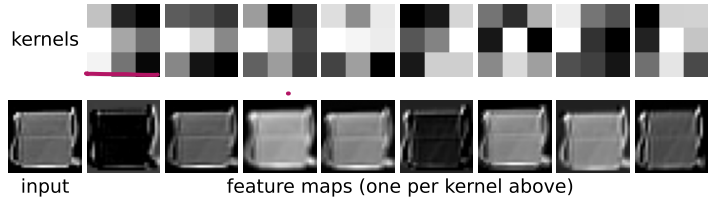
Two things to notice: the **ReLU** matters - without it, stacked convs collapse into one big conv ([41_1]'s linearity lesson); and **pooling has zero parameters**.

# What the first layer learns

Train that CNN on Fashion-MNIST and look at its first-layer kernels (top) and the feature maps they produce (bottom): edge and texture detectors *nobody designed*.

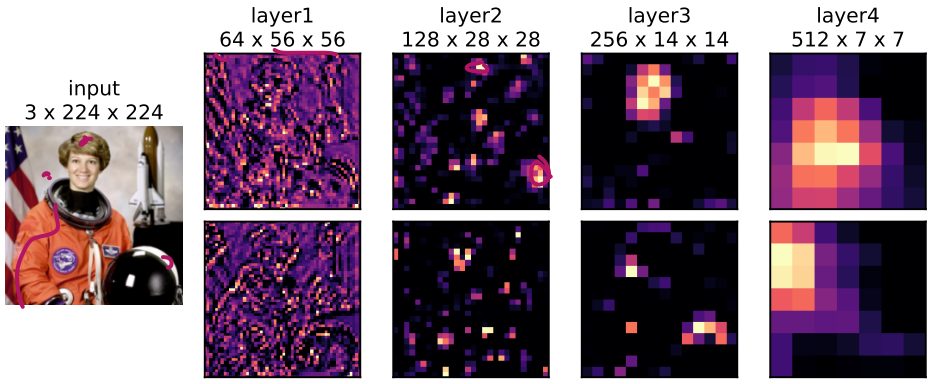
(Trained on ImageNet, first-layer kernels look like textbook edge detectors - L17.)

The first layer is the easy part: it can only see a $3 \times 3$ patch. What do *deeper* layers find?



# Feature maps, layer by layer

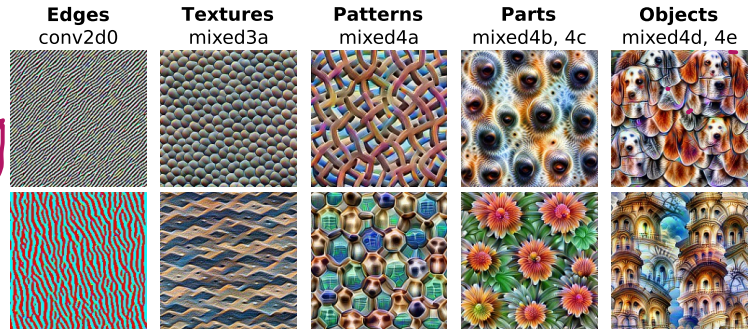
A pretrained ResNet-18 (L17) looks at the astronaut. Two feature maps after each of its four stages (bright = strong response):




Deeper: a **coarser** grid (56 → 7), **more** maps (64 → 512) - less *where*, more *what*. Late maps light up on whole regions: the face, the person.

# What deeper layers look for

**Feature visualization**: change the *input image* until one channel fires as hard as it can - that shows what the channel looks for. Five depths of GoogLeNet (ImageNet):




Edges → textures → patterns → parts → objects, each built from the one below. Nobody told the network these exist: [41_1]'s XOR lesson at scale.

Images: Olah, Mordvintsev & Schubert, "Feature Visualization", Distill (2017), CC BY 4.0.

Go deeper: ▸ Feature Visualization (Distill) ▸ Activation Atlas (Distill) ▸ OpenAI blog ▸ Google blog

# The canonical snippet

The architecture from the anatomy slide, in PyTorch:

```
import torch.nn as nn

net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1),  nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(16 * 7 * 7, 10),
)
# training loop: unchanged from [42]
# forward -> loss -> zero_grad -> backward -> step
```

Autograd differentiates a conv like anything else - [42]'s `loss.backward()` promise (one backward sweep, any graph), kept.

# Why this is fast

A convolution unrolls into a large **matrix multiply**: every output pixel is a dot product, and all of them stack into one big matmul.

That is exactly the shape a **GPU** eats ([42]'s hardware pages) - thousands of identical multiply-adds in parallel.



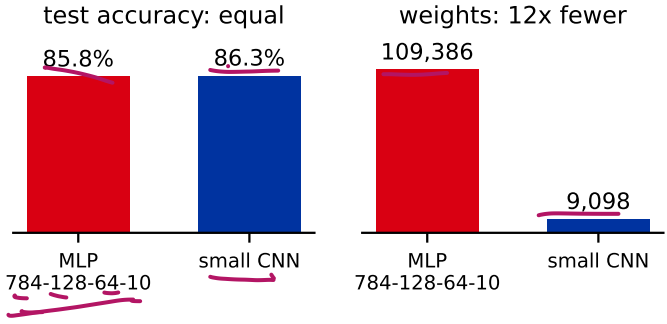
Same reason AlexNet's 60 million parameters trained in under a week on two gaming GPUs in 2012 (next lecture): the operation maps perfectly onto parallel hardware.

# Payoff: CNN vs MLP, same data

Fashion-MNIST ($28 \times 28$ grayscale clothing photos, 10 classes), 15k training images, 15 epochs each: a 784-128-64-10 MLP vs this small CNN.

Accuracy is **about equal**, but the CNN uses $12\times$ **fewer** weights.

The *architecture*, not raw capacity, did the work. In the **Gesture Snake** practical you train a CNN like this on your own webcam photos - and steer a game with it.



## Classical vision we skip

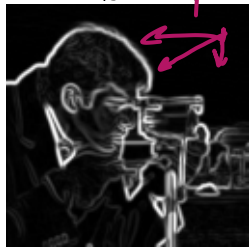
Before CNNs, vision was hand-built from this lecture's kernels. Six classics you will still meet in OpenCV - each one is blur, gradients and one clever idea.

# Canny edge detector (1986)

input



1-2. |gradient|



3. thinned



4. hysteresis



1. **Blur** (Gaussian), or noise becomes edges.
2. **Sobel** X and Y: gradient strength and direction.
3. **Thin**: keep only the maxima along the gradient (1 px wide).
4. **Hysteresis**: strong pixels are edges; weak ones only if they touch a strong one.

John Canny's detector is still the default, `cv2.Canny(img, 50, 150)`: document scanners, measuring parts on a factory line.

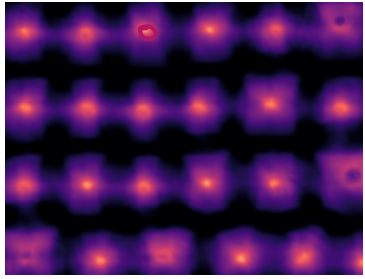
The two thresholds are hand-tuned - per camera, per lighting.

# Template matching

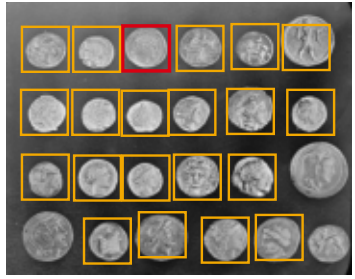
template



score at every position



matches with score > 0.45



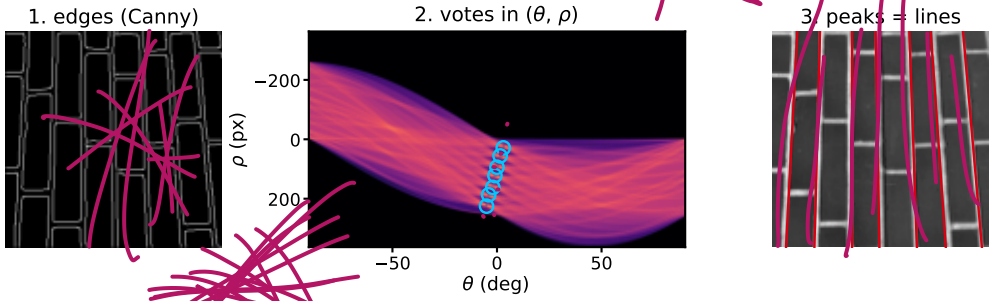
Slide the **template** over the image and score every position by **normalized cross-correlation**: a convolution whose kernel *is* the template, brightness divided out. Peaks are matches.

`cv2.matchTemplate`: UI test bots, circuit-board checks - wherever the target always looks the same.

From one coin it finds 21 of the 24 (score $> 0.45$, red = the template itself).

The 3 it misses include the 2 biggest: it has no idea of **scale** or **rotation**. Change either and the score collapses.

# Hough transform: finding lines by voting




1. Find edges (Canny).
2. Every edge pixel **votes** for every line through it. A line is $x\cos\theta + y\sin\theta = \rho$, so one pixel's votes trace a sinusoid in $(\theta, \rho)$.
3. Where many sinusoids cross, many pixels agree on one line: **peaks are lines**.

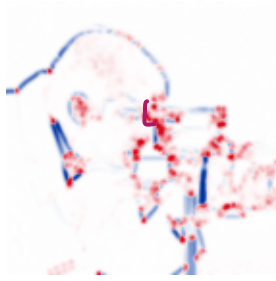
Paul Hough patented it in 1962 to follow particle tracks in bubble-chamber photos; Duda & Hart (1972) gave it the $(\theta, \rho)$ form. `cv2.HoughLines`: lane lines, document edges. The same voting finds circles (`cv2.HoughCircles`): coins, pupils, wheels.

# Harris corner detector (1988)

input



R: red corner, blue edge



52 corners



A **corner** changes in *two* directions. From Sobel gradients $I_x, I_y$, averaged over a small window:

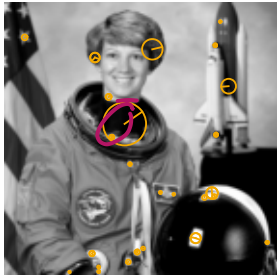
$$M = \sum_{\text{window}} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}, \qquad R = \det M - k\,(\operatorname{tr} M)^2.$$

Two large eigenvalues: corner ($R > 0$). One: edge ($R < 0$). None: flat.

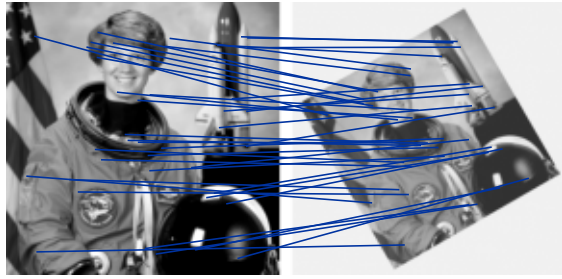
Harris & Stephens, 1988. Corners are the points you can find *again* in the next frame: tracking, video stabilization, AR anchors, panoramas. `cv2.cornerHarris`, or `cv2.goodFeaturesToTrack` (Shi & Tomasi, 1994).

# SIFT: features that survive rotation and zoom

keypoints: where, size, angle



rotated, shrunk, relit copy: 162 of 164 matches correct

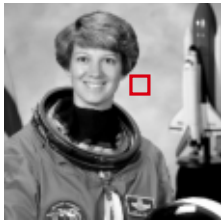


1. Blur at many scales; **keypoints** are blobs that stand out.
2. Each gets its main gradient **angle**: rotation undone.
3. Gradient-angle histograms, $4 \times 4$ cells $\times$ 8 angles: a **128-number fingerprint**.
4. Match fingerprints by nearest neighbour.

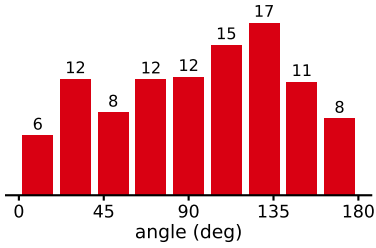
Scale-Invariant Feature Transform (David Lowe, 1999/2004). Panorama stitching, 3D models from photos, robot localization. Patented until March 2020 - now free in `cv2.SIFT_create()`.

# HOG: histograms of oriented gradients (2005)

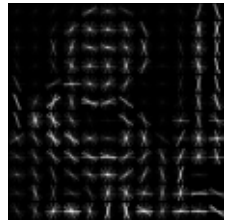
input, one 16x16 cell



its gradient angles, % of strength




every cell's histogram



1. Sobel gradients everywhere.
2. Cells ($8 \times 8$ in the paper, $16 \times 16$ here): each histograms its gradient angles into 9 bins, weighted by strength.
3. Normalize over $2 \times 2$ blocks (lighting), concatenate, and a linear classifier (an SVM) says person / not person.

Dalal & Triggs, 2005 - the pedestrian detector of the 2000s.

HOG is a **hand-built CNN layer**: fixed kernels, binning, pooling, a linear classifier. This whole section is kernels plus hand-made rules; a CNN learns both.

# Recap

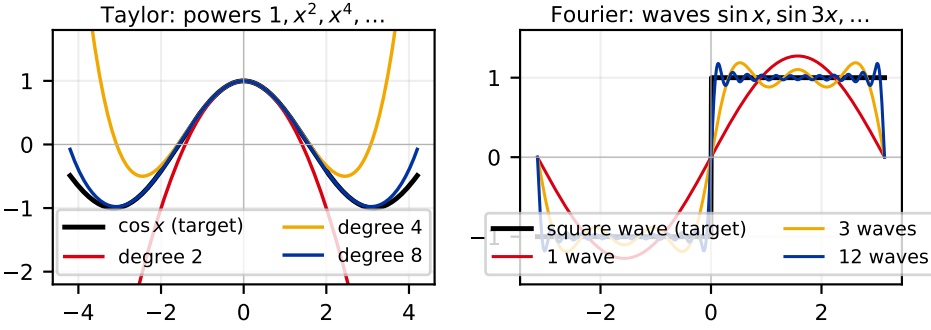
- A dense net cannot afford pixels and ignores their structure.
- Convolution = a **local**, **shared** neuron - a dense layer with most weights forced to 0 and the rest tied: an infinitely strong prior.
- One filter spans the *full input depth* and outputs *one* feature map; $N$ filters $\rightarrow$ $N$ maps. Deeper maps: coarser, more of them, more abstract.
- Two formulas run everything: $o = \lfloor (i - k + 2p)/s \rfloor + 1$ and $\text{params} = k\,k\,C_{\text{in}}C_{\text{out}} + C_{\text{out}}$.
- Pooling has **zero parameters**; it downsamples and buys a little invariance.
- Kernels are **learned**, not hand-designed - a shared weight's gradient is the **sum** over every position it touched.

**Next:** we can build a small CNN. What did 10 years of architecture design buy? The ImageNet story - from 26% error to superhuman (L17).

# Appendix: convolution in frequency space

For the curious: why a kernel is a frequency filter, and why big convolutions are computed with the fast Fourier transform. The full math, with proofs, is in the optional deck *The Convolution, Mathematically*.

# From Taylor to Fourier

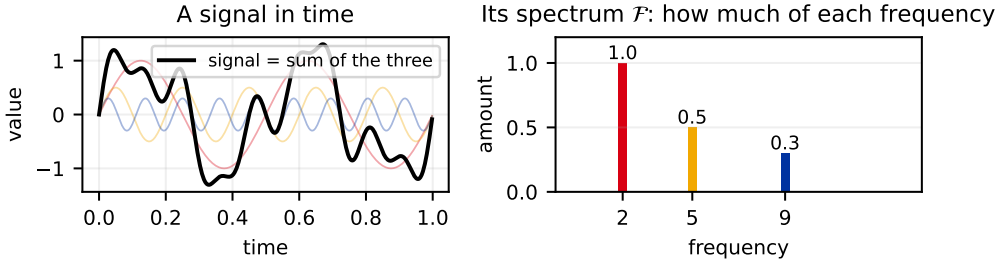



You already know **Taylor**: rebuild a function as a sum of **powers** $1, x, x^2, \ldots$ - a great fit near a point. **Fourier** is the same move with different blocks: a sum of **waves** $\sin x, \sin 2x, \ldots$ of rising *frequency*, and it fits everywhere.

Both break a function into simple pieces. Taylor's are powers; Fourier's are labelled by **frequency**.

# A signal is a bundle of frequencies

The Fourier transform reads off the frequencies hiding in a signal




Add three sine waves and you get one wiggly signal. The **Fourier transform** $\mathcal{F}$ runs it backwards: it reads off *how much of each frequency* sits inside - three tones in, three clean spikes out. That is the signal's **spectrum**.

Same information, two views: the wave in time, or its recipe in frequency.

# Convolution lives in frequency too

The payoff is one clean fact - the **convolution theorem**:

$$\mathcal{F}\{f * g\} \;=\; \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$$

**Convolving in space = multiplying in frequency.** The messy slide-and-sum turns into a plain pointwise product once you switch to the frequency view.

So the fast way to convolve a large image with a large kernel is to take the **fast Fourier transform** (FFT) of both, multiply, and transform back:

$$f * g \;=\; \mathcal{F}^{-1}\big\{\, \mathcal{F}\{f\} \cdot \mathcal{F}\{g\} \,\big\}.$$

A kernel is a **frequency filter**: it scales each frequency of the image up or down.

# Blur and edges are frequency filters

Send the image to frequency space - its **spectrum** (top right: bright centre = low frequencies = smooth areas; outer = high = fine detail and edges).

**Keep the low** frequencies and you get a **blur**. **Keep the high** ones and you are left with **edges**.

You met both as kernels in Section 2: the **box blur** is a low-pass filter, and **Sobel** is a high-pass one.

